

Student 6 **Ashira Bridget Susan S** 192524316RD

4 / 4 submitted

7/29/26, 9:36 PM

Student Submissions — Scenario Based Program Writing

SIMATS Engineering • CSE • CSA0804 — Python Programming • 29-07-2026 • Category: Scenario Based Program Writing • Student Submissions Report

Q1. Intelligent Customer Feedback Analysis

CODE

```
import string

def preprocess_text(text):
    """
    Normalizes input text by converting to lowercase, removing leading/trailing spaces,
    and stripping common punctuation characters.
    """
    # Normalize case and strip whitespace
    cleaned_text = text.lower().strip()

    # Remove standard punctuation using replace()
    for char in string.punctuation:
        cleaned_text = cleaned_text.replace(char, "")

    return cleaned_text

def analyze_sentiment(feedback):
    """
    Analyzes preprocessed feedback text, counts predefined positive and negative keywords,
    and returns the sentiment classification based on keyword frequency.
    """
    # Predefined sentiment keyword lists
    positive_keywords = ["good", "great", "excellent", "amazing", "awesome", "love", "satisfied", "best"]
    negative_keywords = ["bad", "poor", "terrible", "horrible", "worst", "hate", "disappointed", "slow"]

    # Preprocess the feedback text
    cleaned_feedback = preprocess_text(feedback)
    words = cleaned_feedback.split()

    pos_count = 0
    neg_count = 0

    # Count occurrences using loops and conditionals
    for word in words:
        if word in positive_keywords:
            pos_count += 1
        elif word in negative_keywords:
            neg_count += 1

    # Classify sentiment using control flow (if-elif-else)
    if pos_count > neg_count:
        category = "Positive"
    elif neg_count > pos_count:
        category = "Negative"
    else:
        category = "Neutral"

    return category, pos_count, neg_count

def process_multiple_feedbacks(feedback_list):
    """
    Loops through multiple feedback entries and prints analysis results.
    """
    print("--- Feedback Analysis Results ---")
    for idx, entry in enumerate(feedback_list, 1):
        category, pos_cnt, neg_cnt = analyze_sentiment(entry)
        print(f"Entry {idx}: '{entry}'")
        print(f"  → Sentiment: {category} (Positive Keywords: {pos_cnt}, Negative Keywords: {neg_cnt})\n")
```

Student 6 **Ashira Bridget Susan S** 192524316RD

4 / 4 submitted

```
# Main Execution Flow
if __name__ == "__main__":
    # Example handling multiple inputs
    n = int(input("Enter number of feedback entries: "))
    feedbacks = []

    for i in range(n):
        fb = input(f"Enter feedback #{i + 1}: ")

        feedbacks.append(fb)

    process_multiple_feedbacks(feedbacks)
```

INPUT

```
3
The product quality is excellent and I love using it!
Service was terrible, very poor and slow response.
The package arrived yesterday on time.
```

OUTPUT

```
Enter number of feedback entries: Enter feedback #1: Enter feedback #2: Enter feedback #3: --- Feedback Analysis Results ---
Entry 1: 'The product quality is excellent and I love using it!'
-> Sentiment: Positive (Positive Keywords: 2, Negative Keywords: 0)

Entry 2: 'Service was terrible, very poor and slow response.'
-> Sentiment: Negative (Positive Keywords: 0, Negative Keywords: 3)

Entry 3: 'The package arrived yesterday on time.'
-> Sentiment: Neutral (Positive Keywords: 0, Negative Keywords: 0)
```

Student 6 **Ashira Bridget Susan S** 192524316RD

4 / 4 submitted

Q2. Comprehensive Password Strength Checker

CODE

```
def check_password_strength():
    pass_text = input()

    if pass_text == "":
        print("Password Strength: Weak")
        print("Feedback:")
        print("- Password cannot be empty.")
        return

    if len(pass_text) > 128:
        print("Password Strength: Weak")
        print("Feedback:")
        print("- Password is too long (maximum 128 characters allowed).")
        return

    has_upper = False
    has_lower = False
    has_digit = False
    has_symbol = False

    for ch in pass_text:
        if 'A' ≤ ch ≤ 'Z':
            has_upper = True
        elif 'a' ≤ ch ≤ 'z':
            has_lower = True
        elif '0' ≤ ch ≤ '9':
            has_digit = True
        else:
            has_symbol = True

    type_count = 0
    if has_upper:
        type_count = type_count + 1
    if has_lower:
        type_count = type_count + 1
    if has_digit:
        type_count = type_count + 1
    if has_symbol:
        type_count = type_count + 1

    length = len(pass_text)

    if length ≥ 12 and type_count == 4:
        strength = "Very Strong"
    elif length ≥ 8 and type_count ≥ 3:
        strength = "Strong"
    elif length ≥ 6 and type_count ≥ 2:
        strength = "Moderate"
    else:
        strength = "Weak"

    print("Password Strength: " + strength)
    print("Feedback:")
```

Student 6 **Ashira Bridget Susan S** 192524316RD

4 / 4 submitted

```
if length < 8:
    print("- Password should be at least 8 characters long.")
if not has_upper:
    print("- Add at least one uppercase letter.")
if not has_lower:
    print("- Add at least one lowercase letter.")
if not has_digit:
    print("- Add at least one numeric digit.")
if not has_symbol:
    print("- Add at least one special symbol.")

if length ≥ 8 and has_upper and has_lower and has_digit and has_symbol:
    print("- Your password meets all safety criteria!")
```

```
check_password_strength()
```

INPUT

OUTPUT

Student 6 **Ashira Bridget Susan S** 192524316RD

4 / 4 submitted

Q3. Structured Server Log Error Analyser

CODE

```
def extract_and_filter_errors(logs):
    """
    Filters log lines to extract only error entries using string searching methods.
    """
    error_logs = []
    for line in logs:
        line_clean = line.strip()
        # String searching using startswith() or 'in' or find()
        if line_clean.startswith("ERROR") or "ERROR" in line_clean or line_clean.find("[ERROR]") != -1:
            error_logs.append(line_clean)
    return error_logs

def categorize_and_count_errors(error_logs):
    """
    Categorizes types of errors, counts occurrences, and identifies unique errors.
    """
    error_counts = {}
    unique_errors = []

    for err in error_logs:
        # Duplicate removal logic
        if err not in unique_errors:
            unique_errors.append(err)

        # Error categorization and counting logic
        # Categorize based on common error types or default key
        category = "General Error"
        if "Database" in err or "DB" in err:
            category = "Database Error"
        elif "Connection" in err or "Network" in err or "Timeout" in err:
            category = "Network Error"
        elif "Auth" in err or "Permission" in err or "Denied" in err:
            category = "Authentication Error"

        error_counts[category] = error_counts.get(category, 0) + 1

    return error_counts, unique_errors

def generate_summary(error_counts, unique_errors, total_errors):
    """
    Summarizes the analyzed error data.
    """
    print("--- Log Error Analysis Summary ---")
    print("Total Errors Found:", total_errors)
    print("\nError Counts by Category:")
    for category, count in error_counts.items():
        print(f"  - {category}: {count}")

    print("\nUnique Error Messages:")
    for u_err in unique_errors:
        print(f"  * {u_err}")

def process_logs():
    raw_input_data = []
```

Student 6 **Ashira Bridget Susan S** 192524316RD

4 / 4 submitted

```
# Reading input line by line until EOF or blank line
try:
    while True:
        line = input()
        if not line:
            break
        raw_input_data.append(line)
except EOFError:
    pass

# Step 1: Filter Errors
error_logs = extract_and_filter_errors(raw_input_data)

# Step 2: Categorize and Remove Duplicates
error_counts, unique_errors = categorize_and_count_errors(error_logs)

# Step 3: Print Summary Report
generate_summary(error_counts, unique_errors, len(error_logs))

process_logs()
```

INPUT

```
INFO: System startup successful
ERROR: Database connection timeout
WARNING: High memory usage detected
ERROR: Permission denied for user admin
ERROR: Database connection timeout
ERROR: Network socket failed to connect
```

OUTPUT

```
--- Log Error Analysis Summary ---
Total Errors Found: 4

Error Counts by Category:
- Database Error: 2
- Authentication Error: 1
- Network Error: 1

Unique Error Messages:
* ERROR: Database connection timeout
* ERROR: Permission denied for user admin
* ERROR: Network socket failed to connect
```

Student 6 **Ashira Bridget Susan S** 192524316RD

4 / 4 submitted

Q4. Email Domain Frequency Analyser

CODE

```
def extract_domain(email):  
    """  
    Extracts and returns the domain from an email string using split('@').  
    Handles malformed or invalid email addresses gracefully.  
    """  
    email = email.strip()  
  
    # Must contain exactly one '@' symbol and a valid domain part with a dot  
    if email.count('@') != 1:  
        return None  
  
    parts = email.split('@')  
    username = parts[0]  
    domain = parts[1].lower()  
  
    # Basic validation: both username and domain must be non-empty, domain must have '.'  
    if not username or not domain or '.' not in domain:  
        return None  
  
    return domain  
  
def analyze_domain_frequencies(email_list):  
    """  
    Counts frequency of each domain and finds the most frequent domain.  
    """  
    domain_counts = {}  
    invalid_count = 0  
  
    for email in email_list:  
        domain = extract_domain(email)  
        if domain:  
            if domain in domain_counts:  
                domain_counts[domain] = domain_counts[domain] + 1  
            else:  
                domain_counts[domain] = 1  
        else:  
            invalid_count = invalid_count + 1  
  
    # Find domain with maximum frequency using explicit comparisons  
    max_domain = None  
    max_count = 0  
  
    for dom in domain_counts:  
        if domain_counts[dom] > max_count:  
            max_count = domain_counts[dom]  
            max_domain = dom  
  
    return domain_counts, max_domain, max_count, invalid_count  
  
def process_emails():  
    email_data = []
```

Student 6 **Ashira Bridget Susan S** 192524316RD

4 / 4 submitted

```
# Read input until EOF
try:
    while True:
        line = input()
        if not line:
            break
        email_data.append(line)
except EOFError:
    pass

domain_counts, max_domain, max_count, invalid_count = analyze_domain_frequencies(email_data)

print("--- Domain Frequency Report ---")
print("Domain Frequencies:")
for dom in domain_counts:

    print(" - " + dom + ": " + str(domain_counts[dom]))

if max_domain:
    print("\nMost Common Domain: " + max_domain + " (Count: " + str(max_count) + ")")
else:
    print("\nMost Common Domain: None")

if invalid_count > 0:
    print("Invalid/Malformed Emails Encountered: " + str(invalid_count))

process_emails()
```

INPUT

```
alice@gmail.com
bob@yahoo.com
charlie@gmail.com
invalid-email-address
david@company.org
eve@gmail.com
frank@yahoo.com
```

OUTPUT

```
--- Domain Frequency Report ---
Domain Frequencies:
- gmail.com: 3
- yahoo.com: 2
- company.org: 1

Most Common Domain: gmail.com (Count: 3)
Invalid/Malformed Emails Encountered: 1
```